

Mozy

Verified acquisition settlement

Fund demand → Reserve delivery → Deliver normally → Verify → Pay

DOCUMENT PROFILE

CONFIGURATION	attestcoin-spike-cc3-sepolia-2026-08-25
STATUS	implemented testnet release
REFERENCES	architecture.md, attestcoin.md, security.md, runbook.md
OPERATIONS	protocol-runbook.md

Mozy • implemented testnet release

CONTENTS

Whitepaper structure

01	Problem and product thesis
02	Three-statement product model
03	Buyer and solver journey
04	Funded mandates, pricing, and bonds
05	Ordinary foreign delivery invariant
06	Attestcoin verification and semantic checks
07	Creditcoin settlement, replay protection, and authority
08	Architecture and trust relationship
09	Supported deployment and evidence
10	Security assumptions and known limitations

Scope note. This paper describes the implemented testnet release and its explicit security and product boundaries.

1. Problem and product thesis

Organizations often need an asset in a particular operational wallet while their available purchasing capital sits elsewhere. Moving the treasury first can add custody changes, operational handoffs, and chain-specific execution work before the organization has acquired anything. A solver can perform delivery, but a useful market must answer two questions without relying on a coordinator's promise: what payment is actually available, and what evidence is sufficient to release it?

Mozy treats the task as acquisition procurement. A buyer publishes pre-funded demand on Creditcoin. A solver reserves a precise portion, sends the approved asset directly to the buyer's supported-chain wallet, and receives the reserved payment only after the foreign receipt is authenticated and checked by Creditcoin contracts. The delivered asset does not pass through Mozy custody or a Mozy contract on the delivery chain.

The release is deliberately narrow: one configured TEST-for-BTKT testnet market, Ethereum Sepolia delivery, and Creditcoin CC3 settlement. That constraint makes the complete economic path inspectable. It does not imply support for arbitrary assets, chains, or token behavior.

2. Three-statement product model

1. Fund the acquisition on Creditcoin.
2. A solver reserves a fill and sends the asset directly to the buyer's supported-chain wallet.
3. The solver's payment unlocks after delivery is verified.

This yields five visible actions: **Fund demand** → **Reserve delivery** → **Deliver normally** → **Verify** → **Pay**.

Funding and delivery are different commitments. Funding proves that a maximum settlement budget is already in contract custody. Delivery is an ordinary foreign-chain token transfer. Verification makes the foreign receipt usable inside Creditcoin. Settlement is the final on-chain accounting transition; a discovered transfer or a prepared proof is not yet payment.

3. Buyer and solver journey

The buyer creates an Acquisition Mandate on Creditcoin. It identifies the curated market, target TEST quantity, Ethereum Sepolia delivery wallet, price rule, expiry, and reservation duration. The contract computes the maximum funding requirement. The buyer approves and deposits that exact BTKT budget into SettlementVault; only complete funding opens the mandate for reservation.

Open mandates appear as delivery work. A solver inspects the quantity, deterministic price indication, delivery destination, timing rules, and bond before making a commitment. The solver chooses a quantity and requests the current quote. Reservation creation recomputes the quote on-chain so an old or manipulated client quote cannot lock different economics. In one transaction, it binds the solver, removes the quantity from open demand, moves the exact payout from free to reserved budget, and escrows the bond.

The same Ethereum address represents the solver on Creditcoin and Ethereum Sepolia in this release. The solver calls the configured TEST token's ordinary transfer function from that address and sends directly to the mandate's stored delivery wallet. Mozy provides a transaction composer and accepts authenticated registration of an external transaction hash, but neither path changes what qualifies on-chain.

After source discovery, the transaction can remain in `Waiting for verification`. Its receipt may exist before Attestcoin has attested the source block. Once ready, a proof is generated, validated, and submitted to MozySettlement. Only confirmed Creditcoin settlement means BTKT was paid and the solver's bond returned. Because the asset and payout remain on different chains, the solver may still need to rebalance inventory separately; Mozy does not perform that operation.

The buyer can pause or cancel according to mandate lifecycle rules. Cancellation exposes only unreserved capital. An active reservation retains its locked payout and can settle. After the deadline and authenticated source-height grace conditions are both satisfied, anyone may expire an unresolved reservation, returning capacity and payout to the mandate while forfeiting its bond to the buyer.

The interface preserves these distinctions instead of collapsing them into a generic pending state. `Delivery submitted` means a transaction hash has been registered. `Delivery found` means the source transaction and

receipt can be inspected. Waiting for verification means the source block has not completed the Attestcoin readiness path. Proof ready means proof material exists but Creditcoin has not yet confirmed settlement. Settlement submitted identifies an unresolved Creditcoin write. Only Payment confirmed, reconciled with canonical reservation state, completes the economic story. This vocabulary prevents a successful source receipt from being mistaken for a paid solver.

Both parties can later inspect Activity and a deterministic Delivery Receipt. The receipt connects source transaction, token, sender, recipient, observed and credited amounts, proof state, replay identity, reservation, payout, returned bond, and Creditcoin settlement transaction. It is a human-readable projection of public evidence, not a replacement for contract state. If the database is unavailable or rebuilt, the underlying settlement and payout remain unchanged.

4. Funded mandates, pricing, and bonds

An Acquisition Mandate is buyer-defined, pre-funded demand rather than an unenforced quote. SettlementVault separates cumulative history (funded, spent, refunded) from current custody (reserved, free) and maintains:

$$\text{funded} = \text{spent} + \text{reserved} + \text{free} + \text{refunded}$$

The vault's balance must also cover current mandate custody and every unresolved solver bond. Direct transfers to the vault do not credit a mandate. Funding and outgoing token operations measure exact balance changes, which excludes fee-on-transfer and other incompatible behavior from the supported token model.

The release supports deterministic Limit and Range acquisition pricing. All math uses integer base units and rounds down once at the final payout. A reservation quote prices the interval beginning at already acquired plus actively reserved quantity. Contract-side recomputation protects the buyer's curve and the solver's reviewed payout from stale-client disagreement. Full details and the independent reference formula are in [`settlement-model.md`](#).

The reservation bond is 100 basis points of the locked payout with a cap of 10 BTKT. It uses BTKT but remains isolated from the buyer's funded budget. The bond is not a protocol fee: successful settlement returns it to the stored solver. It makes exclusive capacity costly to abandon, while eligible unresolved expiry transfers it to the buyer. This reduces one grieving vector but does not guarantee performance or compensate every possible loss.

5. Ordinary foreign delivery invariant

Mozy deploys no delivery contract or adapter on Ethereum Sepolia. The source transaction targets the configured TEST contract at 0x0F24FD9e0524BA53d3f0A4A40350Adf5370b4A53 and directly encodes `transfer(buyer, amount)`. The qualifying Transfer log is emitted by that same token, with the reserved solver as sender and the stored buyer wallet as recipient.

This invariant matters operationally and semantically. The buyer receives TEST in the requested wallet through standard token behavior. Attestcoin does not move TEST, and the Creditcoin worker never takes custody of it. The foreign receipt is evidence of what Ethereum already executed.

At least the reserved amount must arrive. If a solver overdelivers, settlement still credits only the reserved quantity and releases only the locked payout. This prevents the solver from unilaterally changing the buyer's budget through a larger transfer. A failed receipt, another token, another sender or recipient, insufficient amount, ambiguous qualifying logs, non-transfer calldata, or a source height outside the reservation window does not qualify.

6. Attestcoin verification and semantic checks

The integration uses the current Attestcoin Protocol path with the pinned USC v2 packages @gluwa/usc-contracts@0.1.2 and @gluwa/usc-sdk@0.18.0. On Creditcoin, the official verifier at 0x00FD2 authenticates proof material. ChainInfo at 0x00FD3 supplies authenticated source-height information. The EVM v1 decoder at 0x731c345d79Fb8BbDC541f9DF3b6317585F849F9f exposes receipt fields used by settlement.

When a reservation is created, MozyMarket records an inclusive authenticated source window. For the current deployment, it spans 256 accepted source blocks and adds 64 attested grace blocks before unresolved expiry becomes eligible. This avoids using source RPC timestamps, average block-time guesses, or a relayer-provided clock as economic authority.

Proof generation waits until the proof builder's attested height reaches the transaction block. The proof authenticates the chain key, source height, transaction bytes, Merkle inclusion material, and continuity material. MozySettlement then applies the application-specific meaning. It checks:

- the configured Ethereum Sepolia source chain;
- a successful receipt;
- the TEST transaction target and direct transfer calldata;
- the TEST log emitter;
- reserved solver sender and stored buyer recipient;
- amount at least equal to reserved quantity;
- inclusive reservation source window;
- authenticated transaction and log identity;
- active reservation binding; and
- unused replay identity.

Attestcoin is therefore load-bearing without becoming the payout authority. It makes the foreign receipt verifiable. Mozy's Creditcoin contracts decide whether those authenticated fields satisfy the reservation. A worker can obtain and submit proof material, but cannot make invalid semantics valid.

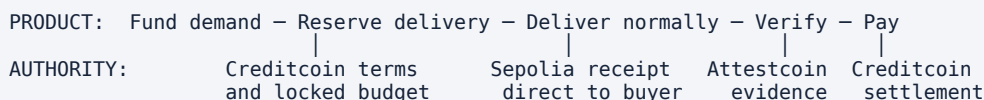
7. Creditcoin settlement, replay protection, and authority

Settlement is permissionless. Any funded relayer may submit the same valid proof, but the destination of value is not a relayer argument. The contract reads the immutable solver and locked payout from the reservation. It derives replay identity from the authenticated source chain key, source height, and transaction index, refuses an already consumed identity, and associates a newly consumed identity with one reservation.

On success, the contract marks the reservation settled, moves the locked payout from reserved to spent accounting, pays the stored solver, returns the isolated bond, updates acquired/reserved quantity, and emits canonical events. State and accounting are updated before external token transfers under reentrancy protection. One foreign receipt cannot pay two reservations.

Creditcoin state is canonical. The Next.js app reads contracts for product actions and renders indexed information for practical discovery. A durable worker, event indexer, reconciler, and Postgres database improve liveness and presentation. Their projections can be rebuilt from the deployment block. Compromise or delay can hide or postpone work, but cannot redirect contract payment or manufacture a settled reservation.

8. Architecture and trust relationship



The web application coordinates wallet intent and presents state. The worker leases candidate jobs, waits for proof readiness, persists checksummed proofs, simulates settlement, submits a network write once, and reconciles known hashes. The indexer tracks canonical events from deployment block 5382887; the reconciler repairs delayed projections. Expiring leases make restarts safe. No component resends a source transfer.

Trust is deliberately split. Ethereum determines whether the TEST transfer executed. Attestcoin authenticates that receipt for Creditcoin. MozySettlement, MozyMarket, and SettlementVault enforce reservation semantics and custody. Users need the availability of RPC and worker infrastructure for timely progress and visibility, but do not grant that infrastructure economic discretion.

9. Supported deployment and evidence

The current market is TEST delivery on Ethereum Sepolia (chain 11155111, source chain key 1) for BTKT settlement on Creditcoin CC3 Testnet (chain 102031). Market ID is 1. Both configured tokens use 18 decimals. Contract addresses, deployment transactions, immutable policies, and compiler settings are recorded in [`../artifacts/protocol/cc3-settlement.json`](#).

The repository commits two independently checkable public evidence manifests:

- [`../artifacts/attestcoin/evidence.json`](#) records the receipt-proof spike through the stateless `ReceiptSemanticProbe`;
- [`../artifacts/protocol/cc3-settlement-evidence.json`](#) records the current proof-to-payment settlement and accounting evidence.

Run `pnpm attestcoin:evidence:check` and `pnpm protocol:evidence:check` to reconcile those artifacts with fresh public chain data. The spike demonstrates the pinned proof path and receipt semantics. The later evidence demonstrates the economic consequence through `MozySettlement`. Neither artifact is a production-security claim.

10. Security assumptions and known limitations

Mozy assumes the safety of Ethereum Sepolia and Creditcoin CC3 consensus for this testnet exercise, the correctness and availability of the pinned Attestcoin path, correct curated configuration, and standard behavior from the configured tokens. It also assumes operators keep disposable private keys and provider credentials separate and secret. Availability of the worker, database, proof builder, and RPC providers affects time to settlement and presentation.

The current release has one configured market, uses one solver Ethereum address across both chains, promises no proof ETA, does not rebalance inventory, supports only the curated standard ERC-20 behavior, and does not increase payout for overdelivery. Contracts are non-upgradeable. The implementation has not been audited or formally verified and is not represented as production-ready.

The threat model, races, key isolation, projection compromise effects, and deferred work are documented in [`security.md`](#). Reproducible setup, health checks, recovery rules, and explorer verification are in [`runbook.md`](#). Those boundaries are part of the product: Mozy makes a narrow verified acquisition path inspectable rather than hiding uncertainty behind a coordinator's claim.

— END OF WHITEPAPER —